INFORME TÉCNICO

AgroSierra
AgroCampo + AgroAPI

Aplicación Android offline-first y API REST segura para labores agrícolas

Estudiante	Gabriel Vasquez Toro
Evaluación	Examen Parcial II - Unidad 2 - Versión A
Fecha	27 de octubre de 2026
Plataforma	Android (única plataforma considerada por la evaluación)
Repositorios previstos	AgroSierra-AgroAPI-Gabriel / AgroSierra-AgroCampo-Gabriel

Resumen

Se preparó una copia local del caso AgroSierra con una aplicación Android en Kotlin Multiplatform y una API propia en Spring Boot, conectada a Oracle mediante Docker Compose. AgroCampo obtiene lotes del servicio, los persiste en SQLDelight y observa los datos locales. En zonas sin red permite registrar labores en cola local y sincronizarlas al recuperar conexión. JWT y privilegios protegen las operaciones.

AgroAPI + Oracle	Contenedores activos; API local localhost:8081; Oracle localhost:1524.
AgroCampo	Ejecutada en emulador Pixel 6 Android; ID pe.upeu.agrosierra.gabriel.
Offline-first	Labor local con ID negativo pendiente; luego sincronizada (1 enviada).
GitHub	Pendiente cambiar/autenticar la cuenta destino; no se ha publicado.
Parte II	El docente asigna SC-A/B/C/D durante la evaluación; aún no se comunicó letra.

Este informe describe el estado verificado localmente; no afirma commits históricos ni publicación todavía no realizada.

1. Arquitectura y alcance

La solución separa dominio, datos y presentación en la app; y seguridad, web, servicios, repositorios y dominio en la API.

AgroAPI	JWT, API REST, autorización y reglas RN-01 a RN-04.	Spring Boot, Security, Oracle, Flyway
AgroCampo/shared	Casos de uso, DTO, errores, ViewModels, SQLDelight y sincronización.	KMP, Ktor, Koin, StateFlow
AgroCampo/androidApp	Pantallas Compose y adaptadores nativos para sesión, base y conectividad.	Android, Compose, Keystore, SQLite
Infraestructura	API y Oracle aislados para esta copia local.	Docker Compose, puertos 8081/1524

Flujo principal

La sincronización envía primero registros pendientes y luego actualiza el padrón local. La pantalla lee de flujos de SQLDelight, no directamente de la respuesta de red. Sin conexión, la labor se guarda con ID negativo y marca pendiente; al sincronizar se asigna el ID definitivo.

Funcionalidad implementada

Autenticación	Credenciales verificadas, JWT, sesión persistente segura, perfil y privilegios.
Lotes y labores	Padrón, filtros/búsqueda, detalle, historial y operaciones CRUD.
Offline-first	Lectura local, guardado provisional, cola y sincronización manual/automática.
Seguridad	Backend exige token y privilegio; no depende de ocultar botones.
Errores	Conversión a errores tipados y mensajes de validación en la interfaz.

2. Reglas de negocio y verificación

Las reglas de negocio se aplican en la capa de servicio y la autorización se hace por privilegios.

RN-01	No registrar/corregir labores en campaña cerrada.	409
RN-02	Técnico corrige las propias solo durante 24 h; jefe puede corregir todas.	403 si incumple
RN-03	No cerrar campaña sin labor Cosecha.	409
RN-04	Fumigación no excede dosis por hectárea × área.	400, campo cantidad

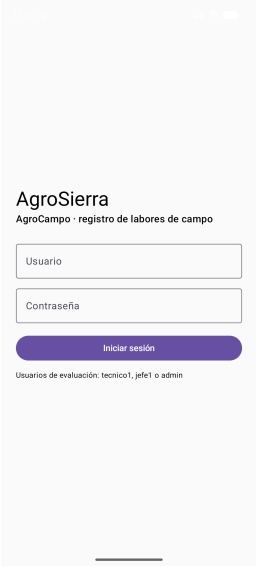
Pruebas del servicio

Credenciales incorrectas / sin token	401 / 401
tecnico1 intenta cerrar campaña	403 Forbidden - No posee el privilegio requerido (CAMPANIA_CERRAR). Petición directa POST /api/v1/lotes/1/cerrar-campania.
Campaña cerrada / cierre sin cosecha	409 RN-01 / 409 RN-03
Fumigación fuera de dosis / editar labor de otro	400 RN-04 / 403 RN-02
Crear y actualizar labor válida	201 / 200
Eliminar por privilegio	Técnico 403; jefe autorizado 204.

Pruebas de código: AgroAPI incluye pruebas unitarias de reglas de labor; AgroCampo commonTest cubre conversión DTO, traducción de errores, registro offline y orden de sincronización. En esta preparación pasaron `mvnw.cmd test` y `gradlew.bat :shared:testAndroidHostTest :androidApp:assembleDebug`.

3. Capturas de la aplicación Android

Capturas del emulador Pixel 6 con AgroCampo-Gabriel instalado, conectado a la API local. La guía de evaluación considera solamente Android.



1. Inicio de sesión



2. Padrón tras autenticación



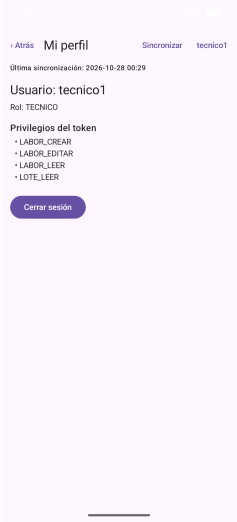
3. Detalle e historial



4. Formulario de labor

4. Evidencia offline, sincronización y autorización

La captura offline muestra una labor local con identificador provisional negativo. Al reconectar, la API la aceptó; después se eliminó el registro temporal para dejar la base de demostración limpia. El padrón actualizado confirma 0 pendientes y 20 registros recibidos.



Perfil con usuario, rol y privilegios



Sin red: labor guardada como pendiente



Padrón sincronizado: 0 enviados y 20 recibidos

Verificación del servicio

La petición directa de cierre de campaña con JWT de **tecnico1** recibió **HTTP 403 Forbidden**: “No posee el privilegio requerido”. La ruta exige **CAMPANIA_CERRAR**.

Estado final limpio

En el padrón se observa “0 enviados y 20 recibidos”. El registro de prueba se quitó del servidor después de verificar el ciclo completo offline/sincronización.

5. Entrega y pendientes

La copia está en el equipo local. Se prevé separar AgroAPI y AgroCampo como dos repositorios, tal como solicita el examen. La cuenta GitHub destino aún no se ha autenticado, por lo que no se crearon los repositorios ni se hizo push.

Backend	AgroSierra-AgroAPI-Gabriel; publicación pendiente de cambiar cuenta.
App Android	AgroSierra-AgroCampo-Gabriel; publicación pendiente de cambiar cuenta.
Etiqueta v1.0-unidad2	No creada porque aún no hay commit evaluado ni historial Git en esta copia.
Ramas y shortlog	No se fabrican fechas, ramas ni commits. El requisito pide historial auténtico distribuido durante la unidad.
Cambio Parte II	Pendiente de la letra SC-A/B/C/D que asigne el docente; no se presenta una alternativa como si fuese la asignada.
Evidencias de ramas/PR	Pendientes hasta que existan ramas, commits y publicación auténticos.

Cambiar la cuenta GitHub

En PowerShell ejecuta primero `gh auth status`. Cierra la sesión anterior con `gh auth logout --hostname github.com --user alexgm-A11`; luego inicia la cuenta nueva con `gh auth login --hostname github.com --web` y verifica de nuevo `gh auth status`. Completa el inicio en el navegador; no pegues contraseñas ni tokens en el chat. Cuando confirmes la nueva cuenta, se crean los dos repositorios, se configuran sus remotos y se sube cada proyecto.

Ejecución local: desde AgroAPI, `docker compose up --build -d`; en el emulador se usa `http://10.0.2.2:8081/api/v1/`. No subir `.env` ni `local.properties`.

Conclusión

La app y el backend están compilados y comprobados localmente, con registro offline y sincronización verificados. La entrega académica aún requiere autenticación/publicación en GitHub, la solicitud SC que asigne el docente y el historial real de ramas y commits.